



Lab 2 - Implementing Decision Instructions in RISC V Assembly Language

Name: Mohammad Husain Naqvi

Student ID: 11062

Task 1:

Code: Listing 3 code:

```
.text
.globl main
main:
    li x22, 1
    li x23, 2
    li x20, 10
    li x21, 5

    bne x22, x23, Else
    add x19, x20, x21
    beq x0, x0, Exit
Else: sub x19, x20, x21

Exit:

end:

j end
```



Listing 4 code:

```
.text
.globl main
main:
li x24, 2 # storing 2 for k =2
li x25, 0x100 #base address for array
li x22, 0 # index i for array
li x5, 2 # for inserting 2 in array
li x4 , 4 # for inserting 4 in array
sw x5 ,0(x25)
sw x5 ,4(x25)
sw x4 ,8(x25)
sw x5 ,12(x25)

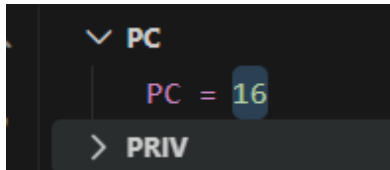
Loop : slli x10,x22,2
      add x10,x10,x25
      lw x9, 0(x10)
      bne x9 ,x24, Exit
      addi x22,x22,1
      beq x0, x0 , Loop

Exit:

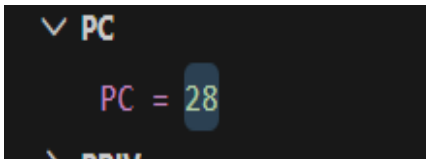
end:
j end
```

**Results:****Listing 3:**

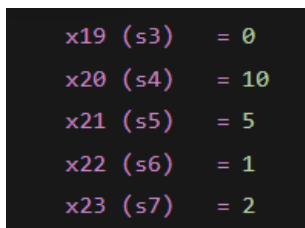
PC before jump:



PC After Jump:



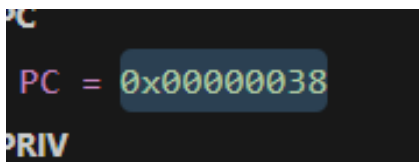
Register Values:



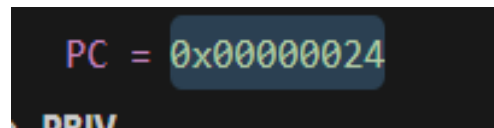
Due to the program
jumping onto Else
condition x19 remains
0.

Listing 4:

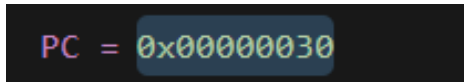
PC before unconditional jump:



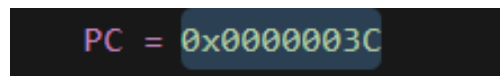
PC after unconditional :



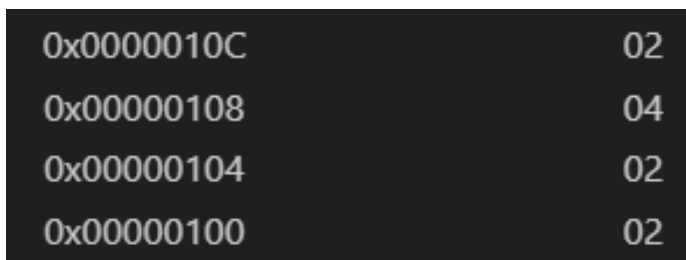
PC before conditional jump:



PC after conditional :



Array Population:



save= [2,2,4,2]

All Register Values : (x25 is the base address)

Integer

x00 (zero) = 0x00000000
x01 (ra) = 0x00000040
x02 (sp) = 0x7FFFFFF0
x03 (gp) = 0x10000000
x04 (tp) = 0x00000004
x05 (t0) = 0x00000002
x06 (t1) = 0x00000000
x07 (t2) = 0x00000000
x08 (s0) = 0x00000000
x09 (s1) = 0x00000004
x10 (a0) = 0x00000108
x11 (a1) = 0x00000000
x12 (a2) = 0x00000000
x13 (a3) = 0x00000000
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000
x16 (a6) = 0x00000000
x17 (a7) = 0x00000000
x18 (s2) = 0x00000000
x19 (s3) = 0x00000000
x20 (s4) = 0x00000000

L/33

x20 (s4) = 0x00000000
x21 (s5) = 0x00000000
x22 (s6) = 0x00000002
x23 (s7) = 0x00000000
x24 (s8) = 0x00000002
x25 (s9) = 0x00000100
x26 (s10) = 0x00000000
x27 (s11) = 0x00000000
x28 (t3) = 0x00000000
x29 (t4) = 0x00000000
x30 (t5) = 0x00000000
x31 (t6) = 0x00000000

2



Task 2:

Code:

```
.text
.globl main

main:

    li x19, 4 # give the value of x

    li x1, 1 # x
    li x2, 2 # x
    li x3, 3 # x
    li x4, 4 # x

    li x21, 2 # a
    li x22, 3 # b
    li x23, 2 # c

    li x10, 2 # immediate 2

    bne x1, x19, Else1

    Case1:
        add x21, x22, x23
        beq x0, x0, Exit

    Else1:
        bne x2, x19, Else2

    Case2:
        sub x21, x22, x23
        beq x0, x0, Exit

    Else2:
        bne x3, x19, Else3

    Case3:
        mul x21, x22, x10
        beq x0, x0, Exit

    Else3:
        bne x4, x19, default

    Case4:
        div x21, x22, x10
        beq x0, x0, Exit

    default:
        li x21, 0

    Exit:

end:

    j end
```

**CASE 1:**

PC before jump:

PC = 0x00000024

Result:

a = b + c: Taking a = 2, b = 2, c = 2

```
x21 (s5) = 0x00000002
x22 (s6) = 0x00000002
x23 (s7) = 0x00000002
```

PC after Jump:

PC = 0x00000028

```
x21 (s5) = 0x00000004
x22 (s6) = 0x00000002
x23 (s7) = 0x00000002
```

CASE 2:

PC before jump:

PC = 0x00000024

Result:

a = b - c: Taking a = 2, b = 2, c = 2

```
x21 (s5) = 0x00000002
x22 (s6) = 0x00000002
x23 (s7) = 0x00000002
```

PC after Jump:

PC = 0x00000034

```
x21 (s5) = 0x00000000
x22 (s6) = 0x00000002
x23 (s7) = 0x00000002
```

CASE 3:

PC before jump:

PC = 0x00000030

Result:

a = b * 2 (Taking a = 2, b = 3, c = 2)

```
x21 (s5) = 0x00000002
x22 (s6) = 0x00000003
x23 (s7) = 0x00000002
```

PC after Jump:

PC = 0x00000040

```
x21 (s5) = 0x00000006
x22 (s6) = 0x00000003
x23 (s7) = 0x00000002
```



CASE :4

PC before jump:

```
PC = 0x0000003C
```

PC after Jump:

```
PC = 0x0000004C
```

Result:

A= b/2 (Taking a= 2 , b = 3, c = 2)

```
x21 (s5) = 0x00000002
x22 (s6) = 0x00000003
x23 (s7) = 0x00000002
```

```
x21 (s5) = 0x00000001
x22 (s6) = 0x00000003
x23 (s7) = 0x00000002
```

Default:

PC before jump

```
PC = 0x00000048
```

PC after jump

```
PC = 0x00000054
```

a= 0 (Taking a= 2 , b = 3, c = 2)

```
x21 (s5) = 0x00000002
x22 (s6) = 0x00000003
x23 (s7) = 0x00000002
```

```
x21 (s5) = 0x00000000
x22 (s6) = 0x00000003
x23 (s7) = 0x00000002
```



Task 3:

Code:

```
.text

.globl main

main:

li x22, 0 # x22 = i

li x11, 0x200 # x11 = a

li x23, 0 # x23 = sum

li x10, 10

For:

    bge x22, x10, Exit # if i >= 10, exit loop

    slli x13, x22, 2 # i * 4

    add x13, x13, x11 # x13 = a + i * 4

    sw x22, 0(x13) # a[i] = i

    addi x22, x22, 1 # i++

    j For # go back to For

Exit:

li x22, 0 # x22 = i

li x11, 0x200 # x11 = a

li x23, 0 # x23 = sum

li x10, 10

For2:

    bge x22, x10, Exit2 # if i >= 10, exit loop

    slli x13, x22, 2 # i * 4

    add x13, x13, x11 # x13 = a + i * 4

    lw x20, 0(x13) # x20 = a[i]

    add x23, x23, x20 # sum += a[i]

    addi x22, x22, 1 # i++

    j For2 # go back to For2

Exit2:

    # Exit label for the second loop

end:

    j end
```



Results:

Register Values at the end of the first loop:

X10= immediate value for loop limit

X11= base address of a = 0x200

X13 = a[i]

X22= i

```
x10 (a0) = 0x0000000A
x11 (a1) = 0x00000200
x12 (a2) = 0x00000000
x13 (a3) = 0x00000224
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000
x16 (a6) = 0x00000000
x17 (a7) = 0x00000000
x18 (s2) = 0x00000000
x19 (s3) = 0x00000000
x20 (s4) = 0x00000000
x21 (s5) = 0x00000000
x22 (s6) = 0x0000000A
```

Register values at the end of the second loop:

```
x10 (a0) = 0x0000000A
x11 (a1) = 0x00000200
x12 (a2) = 0x00000000
x13 (a3) = 0x00000224
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000
x16 (a6) = 0x00000000
x17 (a7) = 0x00000000
x18 (s2) = 0x00000000
x19 (s3) = 0x00000000
x20 (s4) = 0x00000009
x21 (s5) = 0x00000000
x22 (s6) = 0x0000000A
x23 (s7) = 0x0000002D
```

HERE:

X23= sum = 2D (hex)= 45 (decimal) as:

0+1+2+3+4+5+6+7+8+9= 45

X20 = values in a(array)

X22= i

X11= base address a

X13 = base address of a[9]



Task 4:

Code:

```
.text
.globl main

main:

    li x7, 0 # x7 = i
    li x29, 0 # x29 = j
    li x5, 2 # x5 = a
    li x6, 5 # x6 = b
    li x10, 0x100 # x10 = D
    li x4, 4 #temporary
    li x8, 0 # temporary

For1:

    bge x7, x5, Exit # if a >= j, exit loop\

For2:

    bge x29, x6, Exit1 # if b >= j, exit loop

    slli x14, x29, 4 # j *16
    add x14, x14, x10 # x14 = D + j*16
    add x9, x7, x29 # x9 = i + j
    sw x9, 0(x14) # a[j] = i + j

    addi x29, x29, 1 # j++
    beq x0, x0, For2 # go back to For2

Exit1:

    addi x7, x7, 1 # i++
    li x29, 0 # j=0
    beq x0, x0, For1 # go back to For1

Exit:

end:

j end
```

**Results:****Register values at the end: (in decimal)**

X5= a = 2

X6=b = 5

x9 = 5

The last iteration is:

i = 1

j = 4

Therefore:

$x9 = i + j = 1 + 4 = 5$

Then:

sw x9, 0(x14)

stores **5** into the memory location corresponding to D[4*j].

Memory

0x00000100	1	0	0	0
0x00000110	2	0	0	0
0x00000120	3	0	0	0
0x00000130	4	0	0	0
0x00000140	5	0	0	0

Integer

```
x00 (zero) = 0
x01 (ra)    = 76
x02 (sp)    = 2147483632
x03 (gp)    = 268435456
x04 (tp)    = 4
x05 (t0)    = 2
x06 (t1)    = 5
x07 (t2)    = 2
x08 (s0)    = 0
x09 (s1)    = 5
x10 (a0)    = 256
x11 (a1)    = 0
x12 (a2)    = 0
x13 (a3)    = 0
x14 (a4)    = 320
x15 (a5)    = 0
x16 (a6)    = 0
x17 (a7)    = 0
x18 (s2)    = 0
```



GITHUB REPOSITORY LINK:

<https://github.com/MohammadNaqvi1098/CALabs-Git-mn11062.git>



Assessment Rubric

Lab 2 - Implementing Decision Instructions in RISC V Assembly Language

Name: Mohammad Husain	Student ID: 11062	Section: T-4
-----------------------	-------------------	--------------

Points Distribution:

	Task No.	LR 2 (Code)	LR 5 (Results)
In-Lab	Task 1	-	/5
	Task 2	/10	/10
	Task 3	/15	/10
	Task 4	/20	/10
Total Points: 100		/45	/35
CLO Mapped		CLO 2	

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	